Assignment 1: Technique Practice and Prompt Diagnosis

Name: 劉順松
Student ID: 414551016
實驗日期：2026-10-18（Asia/Taipei）。
生成工具： Google Gemini，模式標示 **3.6 Flash**。

Part 1 — Deliberate Technique Practice

Task 1 — Debug get_overdue_tasks

Chain-of-Thought

Prompt (verbatim)

````
task_manager.py:
```python
from datetime import date

def add_task(tasks, title, due_date=None, priority="medium"):
    """Add a new task to the task list.

    Args:
        tasks (list): The list of existing task dictionaries.
        title (str): The task's title.
        due_date (date, optional): The task's due date. Defaults to None.
        priority (str): One of "high", "medium", "low". Defaults to "medium".

    Returns:
        list: The updated list of tasks.
    """
    tasks.append({"title": title, "due_date": due_date, "priority": priority,
"completed": False})
    return tasks

def mark_complete(tasks, title):
    """Mark a task as completed by title.

    Args:
        tasks (list): The list of task dictionaries.
        title (str): The title of the task to mark complete.

    Returns:
        bool: True if a matching task was found and updated, False otherwise.
    """
    for task in tasks:
        if task["title"] == title:
````
````

```
            task["completed"] = True
            return True
    return False

def get_overdue_tasks(tasks, current_date):
    overdue = []
    for task in tasks:
        if task["due_date"] < current_date and not task["completed"]:
            overdue.append(task)
    return overdue

def calculate_completion_rate(tasks):
    if not tasks:
        return 0.0
    completed = sum(1 for t in tasks if t["completed"])
    return completed / len(tasks)
```

Debug get_overdue_tasks. It crashes with TypeError when a task has due_date=None. Give a concise step-by-step execution trace explaining the cause before the corrected function. Preserve the meaning: return incomplete tasks with a date strictly before current_date, and do not modify the input. Return only the short explanation and corrected function.

Full model response

Step-by-Step Execution Trace

1. **Invocation**: `get_overdue_tasks(tasks, current_date)` is called with a list containing a task where `due_date=None`.
2. **Iteration**: The `for` loop evaluates the first task (e.g., `task = {"title": "Task 1", "due_date": None, ...}`).
3. **Condition Evaluation**: Python evaluates `task["due_date"] < current_date`.
4. **Comparison**: Since `task["due_date"]` is `None`, Python attempts to evaluate `None < current_date`.
5. **Crash**: Python raises `TypeError: '<' not supported between instances of 'NoneType' and 'datetime.date'` because `None` cannot be compared to a `date` object using ordering operators.

Corrected Function

```python
def get_overdue_tasks(tasks, current_date):
    overdue = []
    for task in tasks:
        if task["due_date"] is not None and task["due_date"] < current_date and not task["completed"]:
            overdue.append(task)
    return overdue
```

```
```

對話來源：Gemini 實際對話：Debugging Overdue Tasks Function。

Reflection (2–3 sentences)

這個提示要求先追蹤執行，回答因此把錯誤定位到 `None < current_date`，而不只給出修正程式。修正把 `is not None` 放在日期比較之前，該版本通過官方 Task 1 的 4 項檢查；它沒有改變「今天到期不算逾期」的原本語意。

Step-back

Prompt (verbatim)

````
task_manager.py:
```python
from datetime import date

def add_task(tasks, title, due_date=None, priority="medium"):
    """Add a new task to the task list.

    Args:
        tasks (list): The list of existing task dictionaries.
        title (str): The task's title.
        due_date (date, optional): The task's due date. Defaults to None.
        priority (str): One of "high", "medium", "low". Defaults to "medium".

    Returns:
        list: The updated list of tasks.
    """
    tasks.append({"title": title, "due_date": due_date, "priority": priority,
"completed": False})
    return tasks

def mark_complete(tasks, title):
    """Mark a task as completed by title.

    Args:
        tasks (list): The list of task dictionaries.
        title (str): The title of the task to mark complete.

    Returns:
        bool: True if a matching task was found and updated, False otherwise.
    """
    for task in tasks:
        if task["title"] == title:
            task["completed"] = True
            return True
    return False
````
````

```
def get_overdue_tasks(tasks, current_date):
    overdue = []
    for task in tasks:
        if task["due_date"] < current_date and not task["completed"]:
            overdue.append(task)
    return overdue

def calculate_completion_rate(tasks):
    if not tasks:
        return 0.0
    completed = sum(1 for t in tasks if t["completed"])
    return completed / len(tasks)
```

Debug get_overdue_tasks for tasks whose due_date is None. First step back: briefly state the general principles for comparing optional dates and using short-circuit conditions. Then apply those principles to this function and provide the corrected function. Preserve incomplete tasks with dates strictly before current_date, exclude undated tasks, and do not modify the input. Keep the explanation concise.

Full model response

General Principles for Optional Dates and Short-Circuit Evaluation

1. **Avoid Incompatible Comparisons:** In Python, comparing `None` with a `datetime.date` object using relational operators (e.g., `<` or `>`) raises a `TypeError`. Any optional field must be verified as non-`None` before performing a comparison.
2. **Use Short-Circuit Evaluation:** Boolean expressions evaluate left-to-right and stop as soon as the outcome is determined. Placing `task["due_date"] is not None` before the relational check `task["due_date"] < current_date` ensures that Python only evaluates the comparison when `due_date` contains a valid date object.

Application to `get_overdue_tasks`

In `get_overdue_tasks`, tasks created without a `due_date` default to `None`. Evaluating `task["due_date"] < current_date` raises a `TypeError` on those undated tasks.

To fix this while keeping incomplete tasks with dates strictly before `current_date` and excluding undated tasks, update the condition to check that `due_date` is not `None` first:

```python
def get_overdue_tasks(tasks, current_date):
    """Return a list of incomplete tasks whose due date is strictly before current_date.

    Tasks without a due date (due_date is None) are excluded.
    """
```

```
    overdue = []
    for task in tasks:
        if task["due_date"] is not None and task["due_date"] < current_date and
not task["completed"]:
            overdue.append(task)
    return overdue

```
```

對話來源：Gemini 實際對話：Debugging Overdue Tasks Function。

Reflection (2–3 sentences)

Step-back 回答先說明可選日期的型別限制與短路求值，再套用到函式，讓修正有可重用的一般原則。它的實際判斷條件與逐步分析版本相同，也通過 Task 1 的 4 項檢查，因此這次增加的是概念說明，沒有證據顯示功能正確性更高。

Task 2 — Build sort_tasks_by_priority

Zero-shot

Prompt (verbatim)

```
task_manager.py:
```python
from datetime import date

def add_task(tasks, title, due_date=None, priority="medium"):
    """Add a new task to the task list.

    Args:
        tasks (list): The list of existing task dictionaries.
        title (str): The task's title.
        due_date (date, optional): The task's due date. Defaults to None.
        priority (str): One of "high", "medium", "low". Defaults to "medium".

    Returns:
        list: The updated list of tasks.
    """
    tasks.append({"title": title, "due_date": due_date, "priority": priority,
"completed": False})
    return tasks

def mark_complete(tasks, title):
    """Mark a task as completed by title.

    Args:
        tasks (list): The list of task dictionaries.
        title (str): The title of the task to mark complete.

```
```
```

```
    Returns:
        bool: True if a matching task was found and updated, False otherwise.
    """
    for task in tasks:
        if task["title"] == title:
            task["completed"] = True
            return True
    return False

def get_overdue_tasks(tasks, current_date):
    overdue = []
    for task in tasks:
        if task["due_date"] < current_date and not task["completed"]:
            overdue.append(task)
    return overdue

def calculate_completion_rate(tasks):
    if not tasks:
        return 0.0
    completed = sum(1 for t in tasks if t["completed"])
    return completed / len(tasks)
```

Implement sort_tasks_by_priority(tasks). Return a new list of the task dictionaries sorted by priority high, medium, low; within each priority, earlier due dates first and tasks with due_date=None last. Preserve input order for equal keys, do not modify the input list or dictionaries, and return [] for empty input. Assume valid task dictionaries as produced by add_task. Return only the function and any required standard-library imports in one Python code block.

Full model response

```python
from datetime import date

def sort_tasks_by_priority(tasks):
    priority_order = {"high": 0, "medium": 1, "low": 2}

    def sort_key(task):
        p_val = priority_order.get(task["priority"], 3)
        due = task.get("due_date")
        # due_date is either a date object or None
        # None sorts last within the same priority level
        due_key = (1, date.max) if due is None else (0, due)
        return (p_val, due_key)

    return sorted(tasks, key=sort_key)

```

對話來源：Gemini 實際對話：Debugging Overdue Tasks Function。

Reflection (2–3 sentences)

在未提供範例與推理框架的情況下，完整規格已使回答使用優先級排名、日期鍵與 `sorted` 產生新的清單，通過官方 Task 2 的 5 項檢查。以二元日期鍵區分有日期與無日期，也讓補充測試中的實際 `date.max` 排在 `None` 前面；這顯示本次清楚的規格已足以得到正確實作。

Self-correction

零樣本回答直接作為本三輪實驗的第一輪，以下重列原文以保留完整流程。

Turn 1 — Generation prompt and full response

Prompt (verbatim)

````
task_manager.py:
```python
from datetime import date

def add_task(tasks, title, due_date=None, priority="medium"):
    """Add a new task to the task list.

    Args:
        tasks (list): The list of existing task dictionaries.
        title (str): The task's title.
        due_date (date, optional): The task's due date. Defaults to None.
        priority (str): One of "high", "medium", "low". Defaults to "medium".

    Returns:
        list: The updated list of tasks.
    """
    tasks.append({"title": title, "due_date": due_date, "priority": priority,
"completed": False})
    return tasks

def mark_complete(tasks, title):
    """Mark a task as completed by title.

    Args:
        tasks (list): The list of task dictionaries.
        title (str): The title of the task to mark complete.

    Returns:
        bool: True if a matching task was found and updated, False otherwise.
    """
    for task in tasks:
        if task["title"] == title:
            task["completed"] = True
            return True
    return False
````
````

```
def get_overdue_tasks(tasks, current_date):
    overdue = []
    for task in tasks:
        if task["due_date"] < current_date and not task["completed"]:
            overdue.append(task)
    return overdue

def calculate_completion_rate(tasks):
    if not tasks:
        return 0.0
    completed = sum(1 for t in tasks if t["completed"])
    return completed / len(tasks)
```

Implement sort_tasks_by_priority(tasks). Return a new list of the task dictionaries sorted by priority high, medium, low; within each priority, earlier due dates first and tasks with due_date=None last. Preserve input order for equal keys, do not modify the input list or dictionaries, and return [] for empty input. Assume valid task dictionaries as produced by add_task. Return only the function and any required standard-library imports in one Python code block.

Full model response

```
```python
from datetime import date

def sort_tasks_by_priority(tasks):
    priority_order = {"high": 0, "medium": 1, "low": 2}

    def sort_key(task):
        p_val = priority_order.get(task["priority"], 3)
        due = task.get("due_date")
        # due_date is either a date object or None
        # None sorts last within the same priority level
        due_key = (1, date.max) if due is None else (0, due)
        return (p_val, due_key)

    return sorted(tasks, key=sort_key)

```
```

對話來源：Gemini 實際對話：Python Task Priority Sorting Function。

Turn 2 — Critique prompt and full response

Prompt (verbatim)

```
Critique your preceding sort_tasks_by_priority implementation against these
requirements: high/medium/low priority order; ascending dates within each tier;
None last within its tier, including when a dated task has date.max; stable ties;
```

```
a new list with no input mutation; empty input. For each criterion give PASS or
FAIL with a specific reason or counterexample. Distinguish inspection from tests
actually run. Do not invent defects if the code is correct. Then identify any
unnecessary complexity under the valid-input assumption. Do not revise the code
yet.
```

Full model response

```
### Requirement Checklist Critique

Below is an evaluation of the `sort_tasks_by_priority` implementation against each
stated criterion based on code inspection (no unit tests were executed):

1. **High / Medium / Low priority order**
* **Verdict:** PASS
* **Reason:** `priority_order = {"high": 0, "medium": 1, "low": 2}` assigns lower
numerical ranks to higher priorities. In Python's tuple comparison, lower integers
are sorted first (`0 < 1 < 2`), ordering tasks as high, medium, then low.


2. **Ascending dates within each tier**
* **Verdict:** PASS
* **Reason:** For tasks with a valid `date` object, `due_key` evaluates to `(0,
due)`. Python compares `(0, date_A)` vs `(0, date_B)`, where `date_A < date_B`
correctly places earlier dates before later dates.


3. **`None` last within its tier, including when a dated task has `date.max**`
* **Verdict:** PASS
* **Reason:** Tasks with `due_date=None` produce `due_key = (1, date.max)`, while
tasks with a date object produce `due_key = (0, due)`. Tuple comparison evaluates
the first element `0 < 1` before comparing the second element. Even if a task has
`due_date = date.max`, its key is `(0, date.max)`, which sorts before `(1,
date.max)` because $0 < 1$.


4. **Stable ties (preserve input order for equal keys)**
* **Verdict:** PASS
* **Reason:** Python's built-in `sorted()` algorithm (Timsort) is guaranteed to be
stable. Tasks with identical `(priority, due_key)` sort keys preserve their
original relative position.


5. **Return a new list with no input mutation**
* **Verdict:** PASS
* **Reason:** `sorted(tasks, key=sort_key)` constructs and returns a new list of
references without invoking `.sort()` or mutating the dictionary elements.


6. **Empty input (`[]`)**
* **Verdict:** PASS
* **Reason:** Passing `[]` to `sorted()` returns `[]` immediately without
```

````
executing `sort_key`.



---

### Unnecessary Complexity Analysis

Under the assumption of valid input (where `due_date` is always either a `date`
instance or `None`, and `priority` is always `"high"`, `"medium"`, or `"low"`):

1. **Redundant fallback in `priority_order.get()**`
* **Detail:** `.get(task["priority"], 3)` includes a default value of `3`. Under
the valid-input assumption, direct indexing `priority_order[task["priority"]]` is
sufficient.


2. **Redundant second element `date.max` for `None` keys**
* **Detail:** Using a 2-tuple `(0, due)` vs `(1, date.max)` is slightly redundant.
Since the first element (`0` vs `1`) already guarantees that dated tasks sort
before `None` tasks, Python never evaluates the second element of `(1, date.max)`
when comparing against a dated task `(0, due)`. Thus, `(1, None)` or `(1, 0)`
produces identical sorting behavior.
````

對話來源：Gemini 實際對話：Python Task Priority Sorting Function。

Turn 3 — Revision prompt and full response

Prompt (verbatim)

```
Revise the implementation based on that critique. Keep every original requirement,
including None sorting after an actual date.max and stable equal keys. Under the
stated valid-input assumption, remove unnecessary fallbacks and avoid an
unnecessary date import while keeping the code readable. Return only the complete
function in one Python code block.
```

Full model response

````
```python
def sort_tasks_by_priority(tasks):
    priority_order = {"high": 0, "medium": 1, "low": 2}

    def sort_key(task):
        due = task["due_date"]
        due_key = (1, 0) if due is None else (0, due)
        return (priority_order[task["priority"]], due_key)

    return sorted(tasks, key=sort_key)
````
````

```
```

對話來源：Gemini 實際對話：Python Task Priority Sorting Function。

Reflection (2–3 sentences)

具體批評逐項檢查優先級、缺日期、穩定排序與不修改輸入，並明確承認它做的是程式檢視、不是已執行測試。初稿與修訂版都通過官方 5 項排序檢查及額外的 `date.max` 案例，因此修訂的收益是移除有效輸入前提下多餘的預設值與日期匯入，不能宣稱它修好了原本不存在的功能錯誤。

Task 3 — Document calculate_completion_rate

Few-shot

Prompt (verbatim)

````
task_manager.py:
```python
from datetime import date

def add_task(tasks, title, due_date=None, priority="medium"):
    """Add a new task to the task list.

    Args:
        tasks (list): The list of existing task dictionaries.
        title (str): The task's title.
        due_date (date, optional): The task's due date. Defaults to None.
        priority (str): One of "high", "medium", "low". Defaults to "medium".

    Returns:
        list: The updated list of tasks.
    """
    tasks.append({"title": title, "due_date": due_date, "priority": priority,
"completed": False})
    return tasks

def mark_complete(tasks, title):
    """Mark a task as completed by title.

    Args:
        tasks (list): The list of task dictionaries.
        title (str): The title of the task to mark complete.

    Returns:
        bool: True if a matching task was found and updated, False otherwise.
    """
    for task in tasks:
        if task["title"] == title:
            task["completed"] = True
````
````

```
            return True
    return False

def get_overdue_tasks(tasks, current_date):
    overdue = []
    for task in tasks:
        if task["due_date"] < current_date and not task["completed"]:
            overdue.append(task)
    return overdue

def calculate_completion_rate(tasks):
    if not tasks:
        return 0.0
    completed = sum(1 for t in tasks if t["completed"])
    return completed / len(tasks)
```

Use the two provided docstrings of add_task and mark_complete as examples of the documentation style. Write a matching Google-style docstring for calculate_completion_rate, with a one-line summary, Args: and Returns:. Describe the actual return value and the empty-list behavior. Do not change the function logic. Return only calculate_completion_rate with its docstring in one Python code block.

Full model response

````
```python
def calculate_completion_rate(tasks):
    """Calculate the proportion of completed tasks in the task list.

    Args:
        tasks (list): The list of task dictionaries.

    Returns:
        float: The ratio of completed tasks to total tasks as a float between
            0.0 and 1.0, or 0.0 if the task list is empty.
    """
    if not tasks:
        return 0.0
    completed = sum(1 for t in tasks if t["completed"])
    return completed / len(tasks)

```
````

對話來源：Gemini 實際對話：Docstring Generation for Python。

Reflection (2–3 sentences)

兩份指定範例引導回答採用一行摘要、`Args:` 與 `Returns:` 的一致結構，並正確說明回傳 0.0 到 1.0 的比例，以及空清單回傳 0.0。官方文件檢查通過，另以 AST 比對確認只有新增 docstring、未更動計算邏輯；但因提示

也明確指定了格式，不能把格式一致完全歸因於範例本身。

Part 2 — Prompt Diagnosis and Rewrite

測試讀法。 每一題的原始版與改寫版在獨立資料夾執行原封不動的 `python check_my_work.py`。為了隔離問題，資料夾只安裝該題函式，`task_manager.py` 保留老師原始版本；所以完整輸出中其他題目的 SKIP 與 Task 1 已知錯誤，不能算成目前這題的失敗。第二題的頂層匯入錯誤會讓整個 `my_solutions.py` 無法載入，因此它造成多個區塊報 FAIL，根因仍只有該匯入問題。最後另有整合版全部測試。

三題的原始版皆有該題失敗項目，沒有觸發「全部通過時額外重跑兩次」的規則。三題都直接提供了程式，沒有只提問而不寫程式的情況；第二題在提供程式後提出可調整介面的追問，已一併保存。

Prompt 1

Failure mode

缺少團隊規範所造成的介面與資料結構不相容：找不到父任務時回傳 `False` 而不是拋出 `TaskError`，成功時回傳 `True` 而不是原清單，子任務也缺少 `due_date` 與繼承的 `priority`。官方該題結果為 3 PASS、3 FAIL；其中標題正規化的 PASS 是檢查器盲點，補充實驗確認該程式其實沒有正確匹配。

Mechanism

Week 5 的 Context 與 implicit assumptions 說明：模型只能依看到的背景推測專案慣例；原提示詞沒有給它任務錯誤類別、回傳契約與子任務欄位。回覆採用布林成功 / 失敗值，與已提供模組中的 `mark_complete` 模式相似，但僅憑輸出不能證明它內部一定模仿該函式。改寫把五項規範轉成明確 constraints，修正後該題 6 項全數通過；這是補足資訊後的觀察，並非任意換字就改善的證明。

Rewritten prompt

````
task_manager.py:
```python
from datetime import date

def add_task(tasks, title, due_date=None, priority="medium"):
    """Add a new task to the task list.

    Args:
        tasks (list): The list of existing task dictionaries.
        title (str): The task's title.
        due_date (date, optional): The task's due date. Defaults to None.
        priority (str): One of "high", "medium", "low". Defaults to "medium".

    Returns:
        list: The updated list of tasks.
    """
    tasks.append({"title": title, "due_date": due_date, "priority": priority,
"completed": False})
    return tasks

def mark_complete(tasks, title):
    """Mark a task as completed by title.
````
````

```
    Args:
        tasks (list): The list of task dictionaries.
        title (str): The title of the task to mark complete.

    Returns:
        bool: True if a matching task was found and updated, False otherwise.
    """
    for task in tasks:
        if task["title"] == title:
            task["completed"] = True
            return True
    return False

def get_overdue_tasks(tasks, current_date):
    overdue = []
    for task in tasks:
        if task["due_date"] < current_date and not task["completed"]:
            overdue.append(task)
    return overdue

def calculate_completion_rate(tasks):
    if not tasks:
        return 0.0
    completed = sum(1 for t in tasks if t["completed"])
    return completed / len(tasks)
```

Implement add_subtask(tasks, parent_title, subtask_title). Follow these project conventions explicitly: define TaskError as a subclass of Exception in the same file; if the parent is absent raise TaskError, never return False or None as a failure signal. Match titles ignoring case and leading/trailing spaces. Append a new subtask dictionary to the parent's subtasks list, creating that list if absent. The new dictionary must have exactly title=subtask_title, due_date=None, priority equal to the parent's priority, and completed=False. Return the same updated tasks list object. Keep other tasks unchanged. Give the function a Google-style docstring with Args: and Returns: (and document the exception). Assume valid task dictionaries and a list-valued subtasks field when present; on duplicate matching titles use the first match. Return only TaskError and the function in one Python code block.

Evidence — original prompt output

實際送出的原始提示（含完整模組）：

task_manager.py:
```python
from datetime import date

def add_task(tasks, title, due_date=None, priority="medium"):
    """Add a new task to the task list.
```

```
    Args:
        tasks (list): The list of existing task dictionaries.
        title (str): The task's title.
        due_date (date, optional): The task's due date. Defaults to None.
        priority (str): One of "high", "medium", "low". Defaults to "medium".

    Returns:
        list: The updated list of tasks.
    """
    tasks.append({"title": title, "due_date": due_date, "priority": priority,
"completed": False})
    return tasks

def mark_complete(tasks, title):
    """Mark a task as completed by title.

    Args:
        tasks (list): The list of task dictionaries.
        title (str): The title of the task to mark complete.

    Returns:
        bool: True if a matching task was found and updated, False otherwise.
    """
    for task in tasks:
        if task["title"] == title:
            task["completed"] = True
            return True
    return False

def get_overdue_tasks(tasks, current_date):
    overdue = []
    for task in tasks:
        if task["due_date"] < current_date and not task["completed"]:
            overdue.append(task)
    return overdue

def calculate_completion_rate(tasks):
    if not tasks:
        return 0.0
    completed = sum(1 for t in tasks if t["completed"])
    return completed / len(tasks)
```

Write a function add_subtask(tasks, parent_title, subtask_title) that adds a subtask to an existing task.

完整模型回答：

Here is the implementation of `add_subtask`, along with a slight adjustment to `add_task` so that newly created tasks automatically include an empty list for subtasks.

```python
def add_subtask(tasks, parent_title, subtask_title):
    """Add a subtask to an existing parent task by title.

    Args:
        tasks (list): The list of task dictionaries.
        parent_title (str): The title of the parent task.
        subtask_title (str): The title of the subtask to add.

    Returns:
        bool: True if the parent task was found and updated, False otherwise.
    """
    for task in tasks:
        if task["title"] == parent_title:
            # Ensure the subtasks list exists before appending
            if "subtasks" not in task:
                task["subtasks"] = []

            task["subtasks"].append({
                "title": subtask_title,
                "completed": False
            })
            return True
    return False

```

Suggested Update to `add_task`

To keep the data structure consistent across all tasks, update `add_task` to initialize the `subtasks` field when creating a new task:

```python
def add_task(tasks, title, due_date=None, priority="medium"):
    tasks.append({
        "title": title,
        "due_date": due_date,
        "priority": priority,
        "completed": False,
        "subtasks": []
    })
    return tasks

```

對話來源：Gemini 實際對話：Adding Subtasks to Task Manager。

官方檢查器完整輸出：

```
Assignment 1 self-check
This checks your code. It does not give you a grade.
```

```
Part 1, Task 1: get_overdue_tasks (task_manager.py)
--------------------------------------------------
  [FAIL] runs on a list that includes tasks without a due date
         It crashed: TypeError: '<' not supported between instances of 'NoneType'
and 'datetime.date'
         Hint: which line runs first for a task whose due_date is None?

Part 1, Task 2: sort_tasks_by_priority (my_solutions.py)
--------------------------------------------------------
  [skip] sort_tasks_by_priority not found yet
         Paste your function into my_solutions.py to check it.

Part 1, Task 3: calculate_completion_rate docstring (task_manager.py)
---------------------------------------------------------------------
  [ok]   the function still behaves the same (you only add a docstring)
  [skip] docstring not added yet
         Paste your docstring inside the function.

Part 2, Prompt 1: add_subtask (my_solutions.py)
-----------------------------------------------
  [ok]   runs when the parent task exists
  [FAIL] Convention 1: a missing parent task raises TaskError
         Nothing was raised for a task that does not exist.
  [FAIL] Convention 2: returns the updated task list
         It returned bool, not the task list you passed in.
  [ok]   Convention 3: title matching ignores case and surrounding spaces
  [FAIL] Convention 4: the subtask is stored the way the team expects
         subtask is missing the key 'due_date'
         subtask is missing the key 'priority'
  [ok]   Convention 5: Google-style docstring with Args: and Returns:

Part 2, Prompt 2: format_task_line (my_solutions.py)
----------------------------------------------------
  [skip] format_task_line not found yet
         Paste the code for Prompt 2 into my_solutions.py.

Part 2, Prompt 3: get_urgent_tasks (my_solutions.py)
----------------------------------------------------
  [skip] get_urgent_tasks not found yet
         Paste the code for Prompt 3 into my_solutions.py.

Summary
-------
  4 passed, 4 failed, 0 warnings, 4 not started yet
  Read the FAIL lines above. They say what the check saw, not why it happened.
  For Part 2, FAIL lines on the ORIGINAL code are your evidence; keep this output.
  Reminder: a passing check is not the same as a well-diagnosed prompt.
```

補充實驗（非官方檢查器；未更改官方測試）：

```
Supplementary observation: title normalization
Input parent_title: '  write REPORT '
```

```
Returned: False
Parent after call: {'title': 'Write report', 'due_date': datetime.date(2026, 10,
20), 'priority': 'high', 'completed': False}
Subtask actually added: False
Official check reports PASS because it only checks that no exception is raised.
The function returns False without adding a subtask; its PASS is not evidence of
correct matching.
```

Evidence — rewritten prompt output

````
```python
class TaskError(Exception):
    """Exception raised for errors in task management operations."""
    pass


def add_subtask(tasks, parent_title, subtask_title):
    """Add a subtask to an existing parent task.

    Args:
        tasks (list): The list of existing task dictionaries.
        parent_title (str): The title of the parent task to add the subtask to.
        subtask_title (str): The title of the new subtask.

    Returns:
        list: The updated list of tasks.

    Raises:
        TaskError: If no parent task matching `parent_title` is found.
    """
    normalized_parent = parent_title.strip().lower()
    for task in tasks:
        if task["title"].strip().lower() == normalized_parent:
            subtask = {
                "title": subtask_title,
                "due_date": None,
                "priority": task["priority"],
                "completed": False,
            }
            if "subtasks" not in task:
                task["subtasks"] = []
            task["subtasks"].append(subtask)
            return tasks

    raise TaskError(f"Parent task '{parent_title}' not found.")

```
````

對話來源：Gemini 實際對話：Python Task Manager Subtask Implementation。

官方檢查器完整輸出：

```
Assignment 1 self-check
This checks your code. It does not give you a grade.

Part 1, Task 1: get_overdue_tasks (task_manager.py)
--------------------------------------------------
  [FAIL] runs on a list that includes tasks without a due date
         It crashed: TypeError: '<' not supported between instances of 'NoneType'
and 'datetime.date'
         Hint: which line runs first for a task whose due_date is None?

Part 1, Task 2: sort_tasks_by_priority (my_solutions.py)
--------------------------------------------------------
  [skip] sort_tasks_by_priority not found yet
         Paste your function into my_solutions.py to check it.

Part 1, Task 3: calculate_completion_rate docstring (task_manager.py)
---------------------------------------------------------------------
  [ok]   the function still behaves the same (you only add a docstring)
  [skip] docstring not added yet
         Paste your docstring inside the function.

Part 2, Prompt 1: add_subtask (my_solutions.py)
-----------------------------------------------
  [ok]   runs when the parent task exists
  [ok]   Convention 1: a missing parent task raises TaskError
  [ok]   Convention 2: returns the updated task list
  [ok]   Convention 3: title matching ignores case and surrounding spaces
  [ok]   Convention 4: the subtask is stored the way the team expects
  [ok]   Convention 5: Google-style docstring with Args: and Returns:

Part 2, Prompt 2: format_task_line (my_solutions.py)
----------------------------------------------------
  [skip] format_task_line not found yet
         Paste the code for Prompt 2 into my_solutions.py.

Part 2, Prompt 3: get_urgent_tasks (my_solutions.py)
----------------------------------------------------
  [skip] get_urgent_tasks not found yet
         Paste the code for Prompt 3 into my_solutions.py.

Summary
-------
  7 passed, 1 failed, 0 warnings, 4 not started yet
  Read the FAIL lines above. They say what the check saw, not why it happened.
  For Part 2, FAIL lines on the ORIGINAL code are your evidence; keep this output.
  Reminder: a passing check is not the same as a well-diagnosed prompt.
```

Personal connection (one sentence)

我曾請 AI 幫忙新增程式功能，卻只描述「要做什麼」，沒有說明原有程式要求的回傳格式；例如，呼叫端需要取得更新後的清單，AI 卻只回傳 `True` 表示成功，導致功能看似完成，接到其他程式時卻無法按預期使用，這

與本題缺少團隊規範的情況相似。

Prompt 2

Failure mode

未取得工具內容就猜測函式介面：原始程式執行 `from utils import format_date`，但老師提供的工具實際是 `due_label(due, today)`，因此發生 `ImportError`，整個解答模組無法載入。模型已明說這是它的假設，所以應描述為「有揭露但未驗證的工具介面猜測」，不能說它宣稱已讀取 `utils.py`。

Mechanism

Week 6 第 27、32 頁討論 plausible-but-unsupported 輸出與捏造函式介面：常見名稱看似合理，但不能證明私人程式庫真的提供它。這次模型在缺少 context 時以 `format_date` 補空缺，實際匯入失敗提供了外部驗證；改寫透過 grounding 提供真實 `due_label` 邏輯及呼叫規則，四項功能檢查全部通過。原始程式另有大寫優先級及自行產生 "No due date" 的靜態差異，但因模組未載入，不能宣稱原始測試已執行到這兩個分支。

Rewritten prompt

````
task_manager.py:
```python
from datetime import date

def add_task(tasks, title, due_date=None, priority="medium"):
    """Add a new task to the task list.

    Args:
        tasks (list): The list of existing task dictionaries.
        title (str): The task's title.
        due_date (date, optional): The task's due date. Defaults to None.
        priority (str): One of "high", "medium", "low". Defaults to "medium".

    Returns:
        list: The updated list of tasks.
    """
    tasks.append({"title": title, "due_date": due_date, "priority": priority,
"completed": False})
    return tasks

def mark_complete(tasks, title):
    """Mark a task as completed by title.

    Args:
        tasks (list): The list of task dictionaries.
        title (str): The title of the task to mark complete.

    Returns:
        bool: True if a matching task was found and updated, False otherwise.
    """
    for task in tasks:
        if task["title"] == title:
````
````

```
            task["completed"] = True
            return True
    return False

def get_overdue_tasks(tasks, current_date):
    overdue = []
    for task in tasks:
        if task["due_date"] < current_date and not task["completed"]:
            overdue.append(task)
    return overdue

def calculate_completion_rate(tasks):
    if not tasks:
        return 0.0
    completed = sum(1 for t in tasks if t["completed"])
    return completed / len(tasks)
```

The existing utils.py provides the following implementation (docstring abbreviated here; logic unchanged):

```python
def due_label(due, today):
    """Return no due date, due today, due in N day(s), or overdue by N day(s)."""
    if due is None:
        return "no due date"
    delta = (due - today).days
    if delta == 0:
        return "due today"
    if delta > 0:
        return f"due in {delta} day" + ("" if delta == 1 else "s")
    return f"overdue by {-delta} day" + ("" if delta == -1 else "s")
```

Implement format_task_line(task, today). Import and call utils.due_label(task["due_date"], today) for every task, including None dates; do not invent another helper or duplicate its logic. Return one line in the format TITLE [PRIORITY] - DUE_LABEL, preserving the title and lowercase priority exactly and the helper result verbatim. Assume valid task dictionaries with single-line titles. Do not modify the task or utils.py. Include a Google-style docstring with Args: and Returns:. Return only the necessary import and function in one Python code block.

Evidence — original prompt output

實際送出的原始提示（含完整模組）：

task_manager.py:

```python
from datetime import date

def add_task(tasks, title, due_date=None, priority="medium"):
    """Add a new task to the task list.
```

```
    Args:
        tasks (list): The list of existing task dictionaries.
        title (str): The task's title.
        due_date (date, optional): The task's due date. Defaults to None.
        priority (str): One of "high", "medium", "low". Defaults to "medium".

    Returns:
        list: The updated list of tasks.
    """
    tasks.append({"title": title, "due_date": due_date, "priority": priority,
"completed": False})
    return tasks

def mark_complete(tasks, title):
    """Mark a task as completed by title.

    Args:
        tasks (list): The list of task dictionaries.
        title (str): The title of the task to mark complete.

    Returns:
        bool: True if a matching task was found and updated, False otherwise.
    """
    for task in tasks:
        if task["title"] == title:
            task["completed"] = True
            return True
    return False

def get_overdue_tasks(tasks, current_date):
    overdue = []
    for task in tasks:
        if task["due_date"] < current_date and not task["completed"]:
            overdue.append(task)
    return overdue

def calculate_completion_rate(tasks):
    if not tasks:
        return 0.0
    completed = sum(1 for t in tasks if t["completed"])
    return completed / len(tasks)
```

Write a function format_task_line(task, today) that returns a one-line display string for a task, showing its title, priority and due date. Use our existing date-formatting helper in utils.py for the due date.

完整模型回答：

Since `utils.py` wasn't provided in your message, I've assumed a standard date-formatting helper signature (e.g., `format_date(date, today=None)` or similar).

````
Here is the implementation of `format_task_line`:

```python
from utils import format_date

def format_task_line(task, today):
    """Returns a one-line display string for a task showing its title, priority,
and formatted due date.

    Args:
        task (dict): The task dictionary containing 'title', 'priority',
'due_date', and 'completed'.
        today (date): The current date, passed along to the date-formatting helper
if needed.

    Returns:
        str: A formatted string summarizing the task.
    """
    due_str = format_date(task["due_date"], today) if task.get("due_date") else
"No due date"
    status_str = "[x]" if task.get("completed") else "[ ]"

    return f"{status_str} {task['title']} ({task['priority'].upper()}) - Due:
{due_str}"

```

Would you like me to adjust the string layout or import signature to match your
exact utils.py function?
````

對話來源：Gemini 實際對話：Task Formatting Implementation。

官方檢查器完整輸出：

```
Assignment 1 self-check
This checks your code. It does not give you a grade.

Part 1, Task 1: get_overdue_tasks (task_manager.py)
--------------------------------------------------
  [FAIL] runs on a list that includes tasks without a due date
         It crashed: TypeError: '<' not supported between instances of 'NoneType'
and 'datetime.date'
         Hint: which line runs first for a task whose due_date is None?

Part 1, Task 2: sort_tasks_by_priority (my_solutions.py)
--------------------------------------------------------
  [FAIL] sort_tasks_by_priority not found yet
         my_solutions.py has an error when loaded: ImportError: cannot import name
'format_date' from 'utils' (C:\Users\LSS\Documents\1151\1151_535513-計算機架構
\Assignment1\evidence\p2_original_2\utils.py)
         Fix this first: nothing in the file can be checked until it loads.
```

```
Part 1, Task 3: calculate_completion_rate docstring (task_manager.py)
----------------------------------------------------------------------
  [ok]   the function still behaves the same (you only add a docstring)
  [skip] docstring not added yet
         Paste your docstring inside the function.

Part 2, Prompt 1: add_subtask (my_solutions.py)
-----------------------------------------------
  [FAIL] add_subtask not found yet
         my_solutions.py has an error when loaded: ImportError: cannot import name
'format_date' from 'utils' (C:\Users\LSS\Documents\1151\1151_535513-計算機架構
\Assignment1\evidence\p2_original_2\utils.py)
         Fix this first: nothing in the file can be checked until it loads.

Part 2, Prompt 2: format_task_line (my_solutions.py)
----------------------------------------------------
  [FAIL] format_task_line not found yet
         my_solutions.py has an error when loaded: ImportError: cannot import name
'format_date' from 'utils' (C:\Users\LSS\Documents\1151\1151_535513-計算機架構
\Assignment1\evidence\p2_original_2\utils.py)
         Fix this first: nothing in the file can be checked until it loads.

Part 2, Prompt 3: get_urgent_tasks (my_solutions.py)
----------------------------------------------------
  [FAIL] get_urgent_tasks not found yet
         my_solutions.py has an error when loaded: ImportError: cannot import name
'format_date' from 'utils' (C:\Users\LSS\Documents\1151\1151_535513-計算機架構
\Assignment1\evidence\p2_original_2\utils.py)
         Fix this first: nothing in the file can be checked until it loads.

Summary
-------
  1 passed, 5 failed, 0 warnings, 1 not started yet
  Read the FAIL lines above. They say what the check saw, not why it happened.
  For Part 2, FAIL lines on the ORIGINAL code are your evidence; keep this output.
  Reminder: a passing check is not the same as a well-diagnosed prompt.
```

Evidence — rewritten prompt output

```
```python
import utils

def format_task_line(task, today):
    """Format a task dictionary into a single descriptive string line.

    Args:
        task (dict): A task dictionary containing 'title', 'priority', and
'due_date'.
        today (date): The current date used to compute the due status label.

    Returns:
```
```

```
        str: A formatted string in the format TITLE [PRIORITY] - DUE_LABEL.
    """
    label = utils.due_label(task["due_date"], today)
    return f"{task['title']} [{task['priority']}] - {label}"

```

對話來源：Gemini 實際對話：Implementing Task Line Formatting Function。

官方檢查器完整輸出：

```
Assignment 1 self-check
This checks your code. It does not give you a grade.

Part 1, Task 1: get_overdue_tasks (task_manager.py)
--------------------------------------------------
  [FAIL] runs on a list that includes tasks without a due date
         It crashed: TypeError: '<' not supported between instances of 'NoneType'
and 'datetime.date'
         Hint: which line runs first for a task whose due_date is None?

Part 1, Task 2: sort_tasks_by_priority (my_solutions.py)
--------------------------------------------------------
  [skip] sort_tasks_by_priority not found yet
         Paste your function into my_solutions.py to check it.

Part 1, Task 3: calculate_completion_rate docstring (task_manager.py)
---------------------------------------------------------------------
  [ok]   the function still behaves the same (you only add a docstring)
  [skip] docstring not added yet
         Paste your docstring inside the function.

Part 2, Prompt 1: add_subtask (my_solutions.py)
-----------------------------------------------
  [skip] add_subtask not found yet
         Paste the code for Prompt 1 into my_solutions.py.

Part 2, Prompt 2: format_task_line (my_solutions.py)
----------------------------------------------------
  [ok]   runs on tasks with and without a due date
  [ok]   returns a single line of text
  [ok]   each line shows the task's title and priority
  [ok]   the due-date wording is exactly what the team's helper in utils.py
produces

Part 2, Prompt 3: get_urgent_tasks (my_solutions.py)
----------------------------------------------------
  [skip] get_urgent_tasks not found yet
         Paste the code for Prompt 3 into my_solutions.py.

Summary
-------
```

```
  5 passed, 1 failed, 0 warnings, 4 not started yet
  Read the FAIL lines above. They say what the check saw, not why it happened.
  For Part 2, FAIL lines on the ORIGINAL code are your evidence; keep this output.
  Reminder: a passing check is not the same as a well-diagnosed prompt.
```

Personal connection (one sentence)

我曾請 AI 使用現有工具協助撰寫程式，卻沒有提供工具檔案的內容；例如，AI 寫出一個名稱看起來合理的日期格式化函式，我照著執行後才發現無法匯入，檢查原始檔案才知道實際函式名稱不同，讓我理解不能把 AI 猜測的介面直接當成已確認的事實。

Prompt 3

Failure mode

把未定義的「urgent」自行縮限為未完成且今天或以前到期，遺漏未來 1～3 天到期的非高優先任務，也遺漏較晚到期或未指定日期的高優先任務。官方該題為 6 PASS、2 FAIL、1 SKIP；排序檢查因集合不正確而跳過，不能把它寫成已驗證排序失敗。

Mechanism

Week 5 的 Task、Constraints 與 specification-before-generation 適用於此題：原提示詞省略了可以逐項驗證的選取條件與排序規則，模型便自行設定 `due_date <= today`。改寫把條件拆成「未完成 AND（高優先 OR 截止日 <= today+3 天）」並指定跨優先級依日期排序及不修改輸入，九項檢查全數通過。官方檢查器明確納入剛好第 3 天、排除第 4 天的非高優先任務，所以改寫使用這個明確邊界；沒有做多組同義改寫比較，因此不把本次結果稱為已證實的 prompt sensitivity。

Rewritten prompt

````
task_manager.py:
```python
from datetime import date

def add_task(tasks, title, due_date=None, priority="medium"):
    """Add a new task to the task list.

    Args:
        tasks (list): The list of existing task dictionaries.
        title (str): The task's title.
        due_date (date, optional): The task's due date. Defaults to None.
        priority (str): One of "high", "medium", "low". Defaults to "medium".

    Returns:
        list: The updated list of tasks.
    """
    tasks.append({"title": title, "due_date": due_date, "priority": priority,
"completed": False})
    return tasks

def mark_complete(tasks, title):
````
````

```
    """Mark a task as completed by title.

    Args:
        tasks (list): The list of task dictionaries.
        title (str): The title of the task to mark complete.

    Returns:
        bool: True if a matching task was found and updated, False otherwise.
    """
    for task in tasks:
        if task["title"] == title:
            task["completed"] = True
            return True
    return False

def get_overdue_tasks(tasks, current_date):
    overdue = []
    for task in tasks:
        if task["due_date"] < current_date and not task["completed"]:
            overdue.append(task)
    return overdue

def calculate_completion_rate(tasks):
    if not tasks:
        return 0.0
    completed = sum(1 for t in tasks if t["completed"])
    return completed / len(tasks)
```

Implement get_urgent_tasks(tasks, today) with this explicit specification. Exclude all completed tasks. Include an incomplete task if its priority is high OR it has a due date no later than today + timedelta(days=3); this includes all overdue dates, today, and exactly three days ahead. A non-high task due four days ahead is excluded. A high-priority task is included even when due later or undated. Return a new list sorted by earliest due date first across all priorities, placing undated high-priority tasks last, including after an actual date.max. Preserve input order for equal dates. Do not modify the input list or dictionaries. Return [] for empty input. Assume valid task dictionaries with date or None due dates. Include a Google-style docstring with Args: and Returns:. Return only the necessary standard-library imports and function in one Python code block.

Evidence — original prompt output

實際送出的原始提示（含完整模組）：

task_manager.py:
```python
from datetime import date

def add_task(tasks, title, due_date=None, priority="medium"):
    """Add a new task to the task list.
```

```
    Args:
        tasks (list): The list of existing task dictionaries.
        title (str): The task's title.
        due_date (date, optional): The task's due date. Defaults to None.
        priority (str): One of "high", "medium", "low". Defaults to "medium".

    Returns:
        list: The updated list of tasks.
    """
    tasks.append({"title": title, "due_date": due_date, "priority": priority,
"completed": False})
    return tasks

def mark_complete(tasks, title):
    """Mark a task as completed by title.

    Args:
        tasks (list): The list of task dictionaries.
        title (str): The title of the task to mark complete.

    Returns:
        bool: True if a matching task was found and updated, False otherwise.
    """
    for task in tasks:
        if task["title"] == title:
            task["completed"] = True
            return True
    return False

def get_overdue_tasks(tasks, current_date):
    overdue = []
    for task in tasks:
        if task["due_date"] < current_date and not task["completed"]:
            overdue.append(task)
    return overdue

def calculate_completion_rate(tasks):
    if not tasks:
        return 0.0
    completed = sum(1 for t in tasks if t["completed"])
    return completed / len(tasks)
```

Write a function get_urgent_tasks(tasks, today) that returns the urgent tasks,
most urgent first.
```

完整模型回答：

```
To determine which tasks are urgent and how to sort them, we should establish a
clear urgency model based on your task structure:

1. **Urgent Definition:** A task is urgent if it is **uncompleted** (`completed`
```
```

````
is `False`), has a **due date**, and is due **on or before** `today` (i.e.,
overdue or due today).
2. **Sort Order (Most Urgent First):**
* **Due Date (Ascending):** Tasks due earlier (or more severely overdue) come
first. Tasks without a due date are excluded from urgency.
* **Priority Tier (Descending):** If due dates are equal, `"high"` priority
precedes `"medium"`, which precedes `"low"`.



Here is the implementation:

```python
def get_urgent_tasks(tasks, today):
    """Return a list of incomplete tasks due on or before today, sorted by most
urgent first.

    Urgency criteria:
    1. Due date (earlier/overdue dates come first).
    2. Priority ("high" > "medium" > "low").

    Args:
        tasks (list): List of task dictionaries.
        today (date): The reference date to evaluate urgency against.

    Returns:
        list: Sorted list of urgent task dictionaries.
    """
    priority_map = {"high": 0, "medium": 1, "low": 2}

    urgent = [
        t for t in tasks
        if not t.get("completed")
        and t.get("due_date") is not None
        and t["due_date"] <= today
    ]

    urgent.sort(
        key=lambda t: (
            t["due_date"],
            priority_map.get(t.get("priority", "medium"), 1)
        )
    )

    return urgent

```

### Usage Example

```python
from datetime import date

today = date(2026, 10, 18)
````
````

```
tasks = [
    {"title": "Submit report", "due_date": date(2026, 10, 17), "priority":
"medium", "completed": False},
    {"title": "Fix critical bug", "due_date": date(2026, 10, 17), "priority":
"high", "completed": False},
    {"title": "Pay bills", "due_date": date(2026, 10, 18), "priority": "high",
"completed": False},
    {"title": "Buy groceries", "due_date": date(2026, 10, 19), "priority": "high",
"completed": False},  # Future task
    {"title": "Clean desk", "due_date": date(2026, 10, 17), "priority": "low",
"completed": True},     # Already completed
]

urgent_tasks = get_urgent_tasks(tasks, today)
# Output:
# 1. "Fix critical bug" (Oct 17, high)
# 2. "Submit report"     (Oct 17, medium)
# 3. "Pay bills"         (Oct 18, high)

```

對話來源：Gemini 實際對話：Python Urgent Tasks Function Implementation。

官方檢查器完整輸出：

```
Assignment 1 self-check
This checks your code. It does not give you a grade.

Part 1, Task 1: get_overdue_tasks (task_manager.py)
--------------------------------------------------
  [FAIL] runs on a list that includes tasks without a due date
         It crashed: TypeError: '<' not supported between instances of 'NoneType'
and 'datetime.date'
         Hint: which line runs first for a task whose due_date is None?

Part 1, Task 2: sort_tasks_by_priority (my_solutions.py)
--------------------------------------------------------
  [skip] sort_tasks_by_priority not found yet
         Paste your function into my_solutions.py to check it.

Part 1, Task 3: calculate_completion_rate docstring (task_manager.py)
---------------------------------------------------------------------
  [ok]   the function still behaves the same (you only add a docstring)
  [skip] docstring not added yet
         Paste your docstring inside the function.

Part 2, Prompt 1: add_subtask (my_solutions.py)
-----------------------------------------------
  [skip] add_subtask not found yet
         Paste the code for Prompt 1 into my_solutions.py.
```

```
Part 2, Prompt 2: format_task_line (my_solutions.py)
--------------------------------------------------
  [skip] format_task_line not found yet
         Paste the code for Prompt 2 into my_solutions.py.

Part 2, Prompt 3: get_urgent_tasks (my_solutions.py)
--------------------------------------------------
  [ok]   runs on a mixed list of tasks
  [ok]   includes unfinished overdue tasks
  [ok]   leaves out finished tasks
  [FAIL] includes a task due today and one due in exactly 3 days
         Missing: due in 3 days, low
  [ok]   leaves out a task due in 4 days (not high priority)
  [FAIL] includes a high-priority task even when it is due later or has no date
         Missing: due in 10 days, high, no date, high
  [ok]   leaves out an unfinished medium task with no date
  [skip] ordered earliest due date first, undated tasks last
         Fix which tasks are included first; the order check needs the right set.
  [ok]   does not modify the list you passed in

Summary
-------
  7 passed, 3 failed, 0 warnings, 5 not started yet
  Read the FAIL lines above. They say what the check saw, not why it happened.
  For Part 2, FAIL lines on the ORIGINAL code are your evidence; keep this output.
  Reminder: a passing check is not the same as a well-diagnosed prompt.
```

Evidence — rewritten prompt output

```
```python
from datetime import timedelta, date

def get_urgent_tasks(tasks, today):
    """Retrieve and sort urgent tasks that are incomplete.

    Args:
        tasks (list): A list of task dictionaries, each containing 'title',
            'due_date', 'priority', and 'completed' keys.
        today (date): The current reference date.

    Returns:
        list: A new list of urgent task dictionaries sorted by due date, with
            undated high-priority tasks placed last.
    """
    if not tasks:
        return []

    threshold = today + timedelta(days=3)
    urgent_tasks = []

    for task in tasks:
        if task.get("completed"):
```
```

```
            continue

        priority = task.get("priority")
        due_date = task.get("due_date")

        is_high = priority == "high"
        is_due_soon = due_date is not None and due_date <= threshold

        if is_high or is_due_soon:
            urgent_tasks.append(task)

    def sort_key(task):
        due = task.get("due_date")
        if due is None:
            return (1, date.max)
        return (0, due)

    return sorted(urgent_tasks, key=sort_key)
```

對話來源：Gemini 實際對話：Implementing Urgent Tasks Function。

官方檢查器完整輸出：

```
Assignment 1 self-check
This checks your code. It does not give you a grade.

Part 1, Task 1: get_overdue_tasks (task_manager.py)
--------------------------------------------------
  [FAIL] runs on a list that includes tasks without a due date
         It crashed: TypeError: '<' not supported between instances of 'NoneType'
and 'datetime.date'
         Hint: which line runs first for a task whose due_date is None?

Part 1, Task 2: sort_tasks_by_priority (my_solutions.py)
--------------------------------------------------------
  [skip] sort_tasks_by_priority not found yet
         Paste your function into my_solutions.py to check it.

Part 1, Task 3: calculate_completion_rate docstring (task_manager.py)
---------------------------------------------------------------------
  [ok]   the function still behaves the same (you only add a docstring)
  [skip] docstring not added yet
         Paste your docstring inside the function.

Part 2, Prompt 1: add_subtask (my_solutions.py)
-----------------------------------------------
  [skip] add_subtask not found yet
         Paste the code for Prompt 1 into my_solutions.py.

Part 2, Prompt 2: format_task_line (my_solutions.py)
```

```
--------------------------------------------------
  [skip] format_task_line not found yet
         Paste the code for Prompt 2 into my_solutions.py.

Part 2, Prompt 3: get_urgent_tasks (my_solutions.py)
--------------------------------------------------
  [ok]   runs on a mixed list of tasks
  [ok]   includes unfinished overdue tasks
  [ok]   leaves out finished tasks
  [ok]   includes a task due today and one due in exactly 3 days
  [ok]   leaves out a task due in 4 days (not high priority)
  [ok]   includes a high-priority task even when it is due later or has no date
  [ok]   leaves out an unfinished medium task with no date
  [ok]   ordered earliest due date first, undated tasks last
  [ok]   does not modify the list you passed in

Summary
-------
  10 passed, 1 failed, 0 warnings, 4 not started yet
  Read the FAIL lines above. They say what the check saw, not why it happened.
  For Part 2, FAIL lines on the ORIGINAL code are your evidence; keep this output.
  Reminder: a passing check is not the same as a well-diagnosed prompt.
```

Personal connection (one sentence)

我曾請 AI 幫忙安排多項作業的完成順序，只說「先處理緊急的」，卻沒有說明截止日期、重要程度與預估所需時間；例如，AI 把明天到期的小作業排在最前面，卻把幾天後到期但需要較長準備時間的大報告排到後面，讓我發現必須明確說明排序依據，AI 才能更貼近我的需求。

AI Usage Log

工具	請求與使用方式	人工確認狀態
Codex	協助讀取作業、整理知識地圖、設計提示詞、操作瀏覽器、保存回覆、整合程式、執行官方及補充測試、依證據起草反思與診斷並套用範本。	學生提供姓名、學號與材料，明確授權將作業程式及必要規範送至 Gemini；學生亦確認曾經歷三類提示失敗，並指定採用最後提出的三段具體經驗，由 Codex 更新全文；反思與診斷仍需本人閱讀理解。
Google Gemini（介面 Flash 模式）	執行 12 輪生成：Part 1 共 6 輪（零樣本同時是 self-correction 的第一輪），Part 2 共 6 輪；各實驗對話連結附於對應紀錄。	未提供精確後端版本；未偽稱控制溫度或取得模型隱藏推理。
Python 3.12.7	本機執行原始官方檢查器、補充單元測試與 AST / 證據完整性檢查。	程式執行證據已保存；不以模型自評替代實測。

沒有代造模型回答或測試結果。原始失敗、模型多提供的建議與範例、批評中的 Markdown 格式問題均保留；程式複製時僅選取實作區塊，不把示範使用程式誤當解答。最終 `task_manager.py` 採用逐步分析修正與 Few-

shot docstring，`my_solutions.py` 採用排序修訂版與三題改寫版。三句個人經驗採用本人指定的最後版本，描述回傳格式不符、猜錯日期函式及誤排作業順序；未添加專案名稱、日期或工具版本；尚未上傳 E3。

附錄 A：整合驗證

驗證項目	實測結果
官方整合檢查	33 passed、0 failed、0 warnings、0 not started yet
額外邊界與行為測試	8 tests，OK；包含實際新增子任務、日期極大值、穩定排序、工具呼叫、不修改輸入與文件不改邏輯
逐字提示與程式證據核對	12 份提示與 13 個 Python 區塊符合瀏覽器擷取指紋；只忽略區塊首尾空白
官方檢查器完整性	所有實驗資料夾與最終資料夾使用相同原始檢查器 SHA-256

通過測試代表已驗證的案例通過，並非滿分保證，也不代表已涵蓋所有可能輸入。例如日期接近 Python 可表示範圍上限時的加法溢位，並不在本次官方需求測試內。

完整整合輸出：

```
Assignment 1 self-check
This checks your code. It does not give you a grade.

Part 1, Task 1: get_overdue_tasks (task_manager.py)
--------------------------------------------------
  [ok]   runs on a list that includes tasks without a due date
  [ok]   returns exactly the unfinished tasks whose due date is before today
  [ok]   returns [] for an empty list
  [ok]   an unfinished undated task does not hide or break a later overdue task

Part 1, Task 2: sort_tasks_by_priority (my_solutions.py)
--------------------------------------------------------
  [ok]   runs on a list that mixes priorities and missing due dates
  [ok]   order is correct (priority, then due date, undated last in each tier)
  [ok]   does not modify the list you passed in
  [ok]   returns a new list object
  [ok]   returns [] for an empty list

Part 1, Task 3: calculate_completion_rate docstring (task_manager.py)
---------------------------------------------------------------------
  [ok]   the function still behaves the same (you only add a docstring)
  [ok]   starts with a one-line summary
         First line: 'Calculate the proportion of completed tasks in the task
list.'
  [ok]   has an Args: section, like add_task and mark_complete
  [ok]   has a Returns: section, like add_task and mark_complete
  [ok]   documents the tasks parameter

Part 2, Prompt 1: add_subtask (my_solutions.py)
-----------------------------------------------
  [ok]   runs when the parent task exists
```

```
  [ok]   Convention 1: a missing parent task raises TaskError
  [ok]   Convention 2: returns the updated task list
  [ok]   Convention 3: title matching ignores case and surrounding spaces
  [ok]   Convention 4: the subtask is stored the way the team expects
  [ok]   Convention 5: Google-style docstring with Args: and Returns:

Part 2, Prompt 2: format_task_line (my_solutions.py)
--------------------------------------------------
  [ok]   runs on tasks with and without a due date
  [ok]   returns a single line of text
  [ok]   each line shows the task's title and priority
  [ok]   the due-date wording is exactly what the team's helper in utils.py
produces

Part 2, Prompt 3: get_urgent_tasks (my_solutions.py)
--------------------------------------------------
  [ok]   runs on a mixed list of tasks
  [ok]   includes unfinished overdue tasks
  [ok]   leaves out finished tasks
  [ok]   includes a task due today and one due in exactly 3 days
  [ok]   leaves out a task due in 4 days (not high priority)
  [ok]   includes a high-priority task even when it is due later or has no date
  [ok]   leaves out an unfinished medium task with no date
  [ok]   ordered earliest due date first, undated tasks last
  [ok]   does not modify the list you passed in

Summary
-------
  33 passed, 0 failed, 0 warnings, 0 not started yet
  Reminder: a passing check is not the same as a well-diagnosed prompt.
```

補充測試輸出：

```
test_append_preserves_existing_and_missing_parent
(__main__.AdditionalChecks.test_append_preserves_existing_and_missing_parent) ...
ok
test_both_sort_versions_date_max
(__main__.AdditionalChecks.test_both_sort_versions_date_max) ... ok
test_docstring_only_change (__main__.AdditionalChecks.test_docstring_only_change)
... ok
test_format_calls_real_helper_even_for_none
(__main__.AdditionalChecks.test_format_calls_real_helper_even_for_none) ... ok
test_normalized_title_actually_adds
(__main__.AdditionalChecks.test_normalized_title_actually_adds) ... ok
test_sort_date_max_stability_and_no_mutation
(__main__.AdditionalChecks.test_sort_date_max_stability_and_no_mutation) ... ok
test_urgent_boundaries_and_priority
(__main__.AdditionalChecks.test_urgent_boundaries_and_priority) ... ok
test_urgent_ties_and_date_max
(__main__.AdditionalChecks.test_urgent_ties_and_date_max) ... ok

----------------------------------------------------------------------
```

```
Ran 8 tests in 0.006s

OK
```

附錄 B：課程概念依據

- Week 5 `week5-prompt-engineer-1-261005.pdf`：第 34 頁（role/context/task/format/constraints）、第 36–37 頁（模型不知道的背景與慣例）、第 40 頁（implicit assumptions）、第 44–45 頁（specification 與 evaluation）。
- Week 6 `week6-prompt-engineer-2-261012.pdf`：第 27 頁（unsupported / false 的差異）、第 32 頁（不存在的函式介面）、第 34 頁（外部驗證）。
- 本次實作依據 E3 原始範本、`CONVENTIONS.md`、`utils.py` 與未修改的 `check_my_work.py`，並以作業網頁及 PDF 交叉核對要求。